

1. Основные определения: Тестирование ПО, качество ПО, контроль качества, обеспечение качества, ошибки ПО

- Лекция 1 (слайды 1-4, 9-10)

2. Виды тестирования ПО

- Лекция 1 (слайды 11-16 — черный ящик, функциональное тестирование)
- Лекция 2 (принципы тестирования — контекстная зависимость)
- Лекция 3 (белый ящик)
- Лекция 6 (автоматизация)
- Лекция 7 (Unit-тестирование)

3. Основные характеристики качества ПО

- Лекция 1 (слайды 5-8 — модель ISO/IEC 25010:2023)

4. Цели тестирования

- Лекция 1 (слайды 8-9)

5. Принципы тестирования

- Лекция 2

6. Верификация и валидация

- Лекция 1 (QA vs QC, слайды 1-4)
- Лекция 2 (принцип раннего тестирования)
- Лекция 5 (V-образная модель — наглядно показывает верификацию и валидацию на разных уровнях)

7. Тест план, тест дизайн, тестовый сценарий, тест кейс

- Лекция 1 (слайды 10-12 — тестовый дизайн, тест-кейс)
- Лекция 6 (пример тест-кейса TC-LOGIN-01)
- Лекция 7 (структура Unit-теста: [Метод]_[Сценарий]_[Результат])

8. Жизненный цикл разработки ПО

- Лекция 5 (слайд 1)

9. Водопадная модель разработки

- Лекция 5 (слайд 2)

10. V-образная модель разработки

- Лекция 5 (слайд 3)

11. Модель Agile

- Лекция 5 (слайд 4)

12. Автоматизация тестирования

- Лекция 6

13. Пирамида автоматизации

- В лекциях прямо не описана, но вытекает из:
 - Лекция 7 (Unit-тестирование — основание пирамиды)
 - Лекция 6 (UI-тесты — вершина пирамиды)
 - Лекция 3 (белый ящик — нижние уровни)
 - Лекция 1 (черный ящик — верхние уровни)

14. Unit-тестирование

- Лекция 7

15. Функциональное тестирование

- Лекция 1 (слайды 11-14 — пример с расчетом скидки)
- Лекция 6 (пример авторизации)
- Лекция 8 (стр. 4 — черный ящик и функциональные ошибки)

16. Нефункциональное тестирование

- Лекция 1 (слайды 5-8 — модель ISO 25010):
- Лекция 2 (принцип 7 — заблуждение об отсутствии ошибок)

17. Тестирование «белым ящиком»

- Лекция 3

18. Тестирование «черным ящиком»

- Лекция 1 (слайды 11-16 — эквивалентное разбиение, АГС, анализ причинно-следственных связей)
- Лекция 8

19. Usability-тестирование

- Лекция 1 (слайд 6 — Interaction Capability / Способность к взаимодействию)
- Лекция 2 (принцип 7 — заблуждение об отсутствии ошибок)
- В Лекции 8 косвенно (стр. 4 — ошибки в пользовательском интерфейсе)

Вопрос 1. Основные определения: Тестирование ПО, качество ПО, контроль качества, обеспечение качества, ошибки ПО

- **Тестирование ПО (Software Testing):** Это процесс проверки, соответствует ли готовый продукт ожиданиям (требованиям, спецификациям). По сути, это **контроль качества (QC)**. *Подсказка: проверка конечного продукта.*
- **Качество ПО:** Совокупность характеристик, определяющих способность продукта удовлетворять установленные или предполагаемые потребности (ISO 25010). *Подсказка: насколько хорошо продукт выполняет свои функции и удобен.*
- **Контроль качества (QC - Quality Control):** Это процесс проверки готового продукта на наличие дефектов. Тестировщик — это QC-специалист. *Подсказка: действия по обнаружению ошибок в уже созданном продукте.*
- **Обеспечение качества (QA - Quality Assurance):** Это действия на протяжении всего жизненного цикла проекта, чтобы качество было высоким с самого начала. Это про процессы, планирование, ревью кода, метрики. Инженер по QA — это QC + процессы. *Подсказка: предотвращение ошибок на ранних этапах.*
- **Ошибки ПО:** Это причина сбоя. Источники ошибок — от ошибок в требованиях до внешних факторов или взлома. Если ошибка приводит к неправильному поведению, это **дефект (баг)**.

2. Дополнительная информация 2. Дополнительная информация:

- Важно различать QA и QC. QA — про то, как мы строим продукт (процессы), а QC — про сам построенный продукт (проверка).
- Ошибки могут быть обнаружены на любом этапе: в требованиях (самые дорогие для исправления), в дизайне, в коде.

3. Пример

- **QC (Контроль качества):** Тестировщик находит баг, что кнопка "Купить" не работает в корзине.
- **QA (Обеспечение качества):** Команда внедряет практику написания unit-тестов (Лекция 7) и проводит код-ревью, чтобы предотвратить появление таких багов в будущем.

Вопрос 2. Виды тестирования ПО

Виды тестирования можно классифицировать по разным критериям:

- **По уровню (пирамида тестирования, Лекция 7):**
 1. **Unit-тестирование (Модульное):** Тестирование наименьших частей кода (функций, методов).
 2. **Интеграционное:** Проверка взаимодействия между модулями.
 3. **Системное:** Тестирование всей системы в целом.
 - **По доступу к коду и архитектуре (Лекция 1, 3):**
 1. **Тестирование «черным ящиком»:** Без доступа к коду, проверяем только входы и выходы.
 2. **Тестирование «белым ящиком»:** С доступом к коду, проверяем внутренние пути, логику и ветвления.
 - **По типу выполняемых проверок:**
 1. **Функциональное:** Проверяем, корректно ли работают функции продукта (авторизация, оплата, расчет скидки).
 2. **Нефункциональное:** Проверяем, как продукт работает (производительность, удобство использования, безопасность).
 - **По исполнителю:** Ручное (человек) и Автоматизированное (скрипты, Лекция 6).
- ### 2. Дополнительная информация 2. Дополнительная информация:
- На практике тестирование часто бывает смешанным: например, функциональное системное тестирование методом черного ящика.

- Выбор вида тестирования зависит от контекста (Принцип 6, Лекция 2).

3. Пример

- **Unit-тест (белый ящик):** Проверка функции `divide(a, b)` из Лекции 7.
- **Системный тест (черный ящик):** Проверка авторизации (Лекция 6) — вводим email и пароль, смотрим на результат.

Вопрос 3. Основные характеристики качества ПО

Основные характеристики качества по модели ISO/IEC 25010:

1. **Функциональная пригодность (Functional Suitability):** Программа делает то, что должна делать (есть ли нужные функции).
2. **Надежность (Reliability):** Программа стабильно работает без сбоев в заданных условиях.
3. **Способность к взаимодействию (Interaction Capability):** Удобство и понятность интерфейса для пользователя (ранее — юзабилити).
4. **Эффективность производительности (Performance Efficiency):** Соотношение производительности и используемых ресурсов (скорость работы).
5. **Сопровождаемость (Maintainability):** Насколько легко вносить изменения в продукт и исправлять ошибки.
6. **Гибкость (Flexibility):** Возможность переноса в другую среду (ОС, браузер, оборудование).
7. **Совместимость (Compatibility):** Работа с другими системами и устройствами.
8. **Безопасность (Security & Safety):** Защита от взлома и предотвращение рисков для жизни, здоровья или имущества.

2. Дополнительная информация:

- Важно помнить, что это не просто список. Для разных продуктов разные характеристики могут быть приоритетными. Например, для

медицинского ПО важнее безопасность (Safety), а для игр — производительность.

3. Пример

- **Надежность:** Веб-приложение не падает при 1000 одновременных пользователей.
 - **Сопровождаемость:** Разработчик может легко добавить новое правило расчета скидки (из Лекции 1) без риска сломать старые функции.
-

Вопрос 4. Цели тестирования

Цели тестирования:

1. **Предоставление информации для управления рисками:** Показать заказчику и команде, какие есть риски, и насколько продукт готов к выпуску.
2. **Оценка качества продукта:** Понять, оправдал ли продукт ожидания заинтересованной стороны (бизнеса, пользователей).
3. **Оценка корректности исправлений:** Проверить, что найденные дефекты действительно устранены и не вызвали новых проблем.
4. **Оценка корректной реализации изменений:** Убедиться, что новые функции работают правильно и не сломали старые.
5. **Оценка выполнения требований:** Подтвердить, что продукт соответствует всем нормативным, проектным и договорным требованиям.

2. Дополнительная информация:

- Главная цель тестирования — НЕ найти все баги (это невозможно, Принцип 2, Лекция 2), а предоставить информацию о качестве продукта для принятия решения.
- Тестирование — это инструмент управления качеством, а не гарантия его отсутствия.

3. Пример

- После завершения разработки фичи "Промокоды" (Лекция 1), тестирование проводится для того, чтобы ответить на вопрос: "Можно ли выпускать релиз, или риск сломать основной функционал слишком велик?".

Вопрос 5. Принципы тестирования

Семь основных принципов тестирования:

1. **Тестирование показывает наличие дефектов:** Тестирование может найти ошибки, но не гарантирует их полное отсутствие.
2. **Исчерпывающее тестирование невозможно:** Нельзя проверить все комбинации входных данных, путей и условий.
3. **Раннее тестирование:** Чем раньше мы начнем тестировать (например, на этапе требований), тем дешевле будет исправить ошибку.
4. **Скопление дефектов (Принцип Парето):** Около 80% ошибок находится в 20% модулей программы.
5. **Парадокс пестицида:** Если повторять одни и те же тесты снова и снова, они перестают находить новые ошибки. Тесты нужно обновлять.
6. **Тестирование зависит от контекста:** Нельзя применять один и тот же подход для тестирования банковского приложения и компьютерной игры. Методы разные.
7. **Заблуждение об отсутствии ошибок:** Даже если программа работает без ошибок, она может быть бесполезной или неудобной для пользователя. Важно тестировать и нефункциональные требования.

2. Дополнительная информация:

- Принципы — это основа профессии тестировщика. На них нужно ссылаться, объясняя свои решения.
- Принцип "Раннее тестирование" тесно связан с понятием QA (обеспечение качества).

3. Пример

- **Принцип 2 (Невозможность исчерпывающего тестирования):** Вместо проверки всех цен от 1 до 10000 (Лекция 1), мы используем технику "Эквивалентное разбиение", чтобы выбрать представителей из каждого класса.
- **Принцип 5 (Парадокс пестицида):** Если мы каждый релиз проверяем только авторизацию, мы перестанем находить баги, а новые функции останутся не покрытыми.

Вопрос 6. Верификация и валидация

Это два ключевых процесса проверки качества, которые часто путают:

- **Верификация (Verification):** "Мы строим продукт правильно?". Это процесс проверки, соответствует ли продукт зафиксированным требованиям, спецификациям, стандартам. Часто это проверка документации, кода, дизайна. Не требует запуска программы.
- **Валидация (Validation):** "Мы строим правильный продукт?". Это процесс проверки, соответствует ли продукт реальным потребностям и ожиданиям пользователя. Здесь мы запускаем продукт и смотрим, решает ли он задачу пользователя.

2. Дополнительная информация:

- Простая мнемоника: Верификация — ответ на вопрос "Сделали ли мы это по чертежам?", Валидация — "Хотят ли люди такой дом?".
- Верификация — это часть QA (обеспечение качества). Валидация — это часть QC (контроль качества). Верификация может включать в себя ревью кода (code review), а валидация — функциональное тестирование.

3. Пример

- **Верификация:** Проверка того, что в коде правильно реализованы все классы эквивалентности для расчета скидки из Лекции 1 (все if/else соответствуют спецификации). Это можно сделать, не запуская код.
 - **Валидация:** Вы запускаете готовый интернет-магазин и проверяете, что итоговая цена со скидкой понятна пользователю и соответствует его ожиданиям от акции.
-

Вопрос 7. Тест план, тест дизайн, тестовый сценарий, тест кейс

Иерархия артефактов тестирования:

1. **Тест-план (Test Plan):** Это стратегический документ. Он описывает, что и как мы будем тестировать, какие ресурсы нужны, сроки, риски и подходы. *Подсказка: "Глобальный план всей проверки".*
2. **Тестовый дизайн (Test Design):** Это процесс создания тестовых сценариев на основе требований и выбранных техник (например, эквивалентное разбиение). Цель — покрыть максимум сценариев минимальным числом тестов. *Подсказка: "Как именно мы будем создавать тесты, чтобы они были эффективными".*
3. **Тест-кейс (Test Case):** Это конкретная артефакт. Детальная инструкция, состоящая из: ID, предусловий, шагов и ожидаемого результата. *Подсказка: "Готовая инструкция для проверки одного сценария".*
4. **Тестовый сценарий (Test Script):** Часто это синоним тест-кейса, но в контексте автоматизации — это готовый к исполнению скрипт (код). В контексте ручного тестирования — это совокупность шагов в тест-кейсе. *Подсказка: "Сценарий — это шаги, которые описывают, что делать".*

2. Дополнительная информация:

- Тест-план отвечает на вопрос "ЗАЧЕМ и КОГДА мы тестируем", тест-дизайн — "ЧТО мы будем тестировать и почему", а тест-кейс — "КАК именно мы проверим один конкретный пункт".

3. Пример

- **Тест-план:** "С 1 по 5 октября мы тестируем модуль оплаты на тестовом стенде".
- **Тестовый дизайн:** Используя анализ граничных значений, мы решаем проверить цену товара price на границах 1 и 10000 (из Лекции 1).
- **Тест-кейс (TC-LOGIN-01 из Лекции 6):** Это и есть готовый пример. В нем есть ID, предусловия, шаги и ожидаемый результат.

Вопрос 8. Жизненный цикл разработки ПО

Жизненный цикл приложения (SLCM) — это весь период времени от идеи до полного снятия продукта с эксплуатации.

Жизненный цикл разработки ПО (SDLC) — это применение стандартных бизнес-практик для создания приложений. Это процесс, который проходит разработка.

Основные этапы (общие для всех моделей): Анализ требований -> Проектирование -> Разработка (кодинг) -> Тестирование -> Внедрение (развертывание) -> Эксплуатация и поддержка.

2. Дополнительная информация:

- Важно понимать разницу между SLCM и SDLC. SDLC — это часть SLCM, которая касается именно процесса разработки.
- Существует несколько моделей SDLC (о них следующие вопросы): Водопад, V-образная, Agile, Спиральная.

3. Пример

- **SLCM:** Для приложения "Калькулятор скидок" жизненный цикл начался с идеи бизнеса, прошел этапы разработки и тестирования

(Лекция 1), а закончится, когда компания решит прекратить его поддержку.

- **SDLC:** Конкретный процесс, по которому команда разработчиков создала этот калькулятор.

Вопрос 9. Водопадная модель разработки

Водопад (Waterfall) — это классическая, линейная и последовательная модель разработки.

Суть: Этапы выполняются строго один за другим: **Анализ -> Проектирование -> Разработка -> Тестирование -> Внедрение -> Поддержка**. Переход на следующий этап возможен только после полного завершения предыдущего.

Особенности: Требования фиксируются в самом начале, и в процессе разработки их сложно менять.

2. Дополнительная информация:

- **Преимущества:** Простота управления, четкие этапы и сроки.
- **Недостатки:** Обнаружение серьезных ошибок на поздних этапах (на тестировании) обходится очень дорого (Принцип "Раннее тестирование", Лекция 2). Не подходит для проектов с меняющимися требованиями.
- Сейчас почти не используется в чистом виде в IT, но часто преподается как базовая историческая модель.

3. Пример

- Разработка государственной системы учета, где требования строго регламентированы и не меняются. Всех разработчиков можно собрать в комнате, утвердить требования и начать разработку "по этапам".
-

Вопрос 10. V-образная модель разработки

V-образная модель (V-Model) — это эволюция водопада, в которой процесс тестирования интегрирован на всех этапах разработки.

На схеме она выглядит как буква "V". Слева вниз идет разработка (этапы детализации требований и проектирования), а справа вверх — тестирование (этапы верификации и валидации). Каждый этап разработки имеет соответствующий этап тестирования.

1. **Анализ требований** <-> **Приемочное тестирование** (проверяем у заказчика)
2. **Системное проектирование** <-> **Системное тестирование** (проверяем систему целиком)
3. **Архитектурное проектирование** <-> **Интеграционное тестирование** (проверяем связи между модулями)
4. **Модульное проектирование (Кодинг)** <-> **Модульное тестирование (Unit-тесты)** (проверяем отдельные функции)

2. Дополнительная информация:

- **Ключевое преимущество:** Тестирование не откладывается на конец, а планируется параллельно с разработкой. Это позволяет выявлять ошибки на более ранних стадиях.
- **Недостаток:** Как и водопад, она остается довольно жесткой и не очень гибкой к изменениям требований.

3. Пример

- Когда разработчик пишет функцию `divide(a, b)` (Лекция 7), он параллельно или даже заранее пишет для нее Unit-тесты (это соответствует правой ветке V-модели). Когда системный архитектор проектирует модули, тестировщик проектирует интеграционные тесты.

Вопрос 11. Модель Agile

Agile (Гибкая методология) — это не одна модель, а целая философия и набор подходов к разработке, основанных на итеративности и инкрементальности.

Ключевые идеи:

- Разработка ведется **короткими циклами (итерациями / спринтами)**, обычно 1–4 недели.
- В конце каждого спринта получается **работающий продукт** с новыми функциями.
- Требования могут меняться в течение проекта, и это приветствуется.
- Тесное взаимодействие с заказчиком и внутри команды.

Основные фреймворки Agile:

- **Scrum:** Работа ведется в спринтах, есть четкие роли (Product Owner, Scrum Master, команда) и артефакты (бэклог, доска задач).
- **Kanban:** Визуализация потока задач на доске, нет строгих спринтов, акцент на непрерывную доставку и управление WIP (Work In Progress).

2. Дополнительная информация:

- Agile — это ответ на недостатки "тяжелых" моделей вроде Водопада, где изменения требований были крайне дорогими и болезненными.
- В Agile тестирование — не отдельный этап в конце, а **непрерывный процесс**. Тестировщики участвуют в планировании, пишут автотесты параллельно с разработкой.
- Обратная связь от пользователей получается очень быстро, что позволяет корректировать курс.

3. Пример

- Команда разрабатывает интернет-магазин. Первый спринт: реализовали и протестировали авторизацию (как в Лекции 6). Второй спринт: добавили каталог товаров. Третий: систему скидок (как в Лекции 1).

Заказчик видит работающий продукт после каждого спринта и может вносить правки.

Вопрос 12. Автоматизация тестирования

Автоматизация тестирования — это процесс использования специализированных программных средств (скриптов, фреймворков) для выполнения тестовых сценариев вместо ручного выполнения.

Что автоматизируют:

- Повторяющиеся тесты (например, регрессионные).
- Нагрузочные тесты (имитация множества пользователей).
- Тесты, которые сложно или долго выполнять вручную (например, проверка большого количества комбинаций входных данных).

Из примера в лекции (Лекция 6):

- Сценарий авторизации описан шагами (открыть страницу, ввести email, ввести пароль, нажать кнопку).
- В автоматизации эти шаги превращаются в код, который эмулирует действия пользователя в браузере.

2. Дополнительная информация:

- Автоматизация **не заменяет** ручное тестирование. Ручное тестирование лучше подходит для исследовательского тестирования, проверки юзабилити и нововведений.
- Автотесты требуют поддержки: если код меняется, тесты нужно обновлять (иначе — "парадокс пестицида" и ложные срабатывания).
- Примеры инструментов: Selenium, Cypress, JUnit, PyTest.

3. Пример

- Вместо того чтобы вручную заходить в систему с разными логинами и паролями (TC-LOGIN-01, 02, 03 из Лекции 6), тестировщик пишет скрипт, который делает это автоматически каждый раз после сборки нового релиза, проверяя, что авторизация не сломалась.

Вопрос 13. Пирамида автоматизации

Пирамида автоматизации (Test Automation Pyramid) — это стратегия распределения автотестов по уровням, предложенная Майком Коном.

Уровни пирамиды (снизу вверх):

1. **Основание (Unit-тесты):** Много. Самые быстрые, дешевые, стабильные. Проверяют отдельные модули и функции (как в Лекции 7). Их должно быть **больше всего**.
2. **Средний уровень (API / Интеграционные тесты):** Среднее количество. Проверяют взаимодействие между компонентами, сервисами, базами данных. Быстрее, чем UI-тесты.
3. **Верхушка (UI / E2E тесты):** Мало. Самые медленные, дорогие и хрупкие. Проверяют пользовательские сценарии через интерфейс (как авторизация в Лекции 6). Их должно быть **меньше всего**.

2. Дополнительная информация:

- Суть пирамиды: чем выше уровень, тем тесты дороже в написании и поддержке, и тем их должно быть меньше.
- Если у вас пирамида "перевернута" (много UI-тестов и мало Unit-тестов), это плохая стратегия — тесты будут долго выполняться и часто ломаться.
- Важно помнить принцип: "Надежный фундамент — быстрые и точные Unit-тесты".

3. Пример

- Для функции расчета скидки (Лекция 1):
 - **Unit-тест:** Проверяем математику расчета цены со скидкой в отдельной функции (быстро, много вариантов).
 - **API-тест:** Проверяем, что эндпоинт `/calculate` на бэкенде возвращает правильный JSON.

- **UI-тест:** Проверяем, что пользователь видит на экране итоговую цену после всех действий. Таких тестов делаем всего пару штук на ключевые сценарии.

Вопрос 14. Unit-тестирование

Unit-тестирование (Модульное тестирование) — это проверка мельчайших, изолированных частей кода (функций, методов, классов) на корректность работы.

В лекции приведен отличный пример с функцией `divide(a, b)`:

- Проверяем обычные числа: `divide(10, 2) == 5`.
- Проверяем деление на ноль: перехватываем исключение `ZeroDivisionError`.
- Проверяем граничные и отрицательные значения: `divide(1, 10000)`, `divide(-10, 2)`, `divide(0, 5)`.

Структура

названия

тест-кейса:

`[ТестируемыйМетод]_[Сценарий]_[ОжидаемыйРезультат]`

Например: `Sum_TwoNumbers_ReturnsSum()`

2. Дополнительная информация:

- Unit-тесты пишутся разработчиками (иногда и тестировщиками) и запускаются часто, почти после каждого изменения кода.
- Это первый и самый быстрый барьер, защищающий от регрессий.
- В хорошей практике, часть тестов пишется **до** написания кода (TDD — Test-Driven Development).
- В лекции подчеркивается, что тестирование должно покрывать все независимые пути в модуле.

3. Пример

- Из лекции 7: `test_edge_cases()` проверяет, что произойдет, если цена будет равна 0, а не только стандартным 5 или 10. Без такого теста

разработчик мог бы забыть про ноль, и в продакшене возникла бы ошибка.

Вопрос 15. Функциональное тестирование

Функциональное тестирование — это проверка того, *что* делает система. Оно направлено на проверку соответствия программы зафиксированным функциональным требованиям.

Цель: убедиться, что каждая функция приложения работает именно так, как ожидается.

Пример из лекции (расчет скидки):

- Проверяем корректные сценарии: пользователь "registered" + промокод "SALE10" -> скидка 15% (5% + 10%).
- Проверяем граничные значения: цена 1 и 10000.
- Проверяем ошибки: цена < 1 или > 10000 -> сообщение об ошибке; неизвестный промокод -> сообщение об ошибке.

2. Дополнительная информация:

- Функциональное тестирование тесно связано с техниками "черного ящика" (Лекция 1): нас не интересует, как устроен код внутри, мы проверяем входные данные и выходные результаты.
- Оно отвечает на вопрос: "Работает ли кнопка 'Купить'?".
- Виды функционального тестирования: Smoke-тесты, Санитарные, Регрессионное, Тестирование новой функциональности.

3. Пример

- В примере с авторизацией (Лекция 6) функциональные тесты: проверить вход с верным логином (успех), с неверным паролем (ошибка), с пустыми полями (предупреждение).
-

Вопрос 16. Нефункциональное тестирование

Нефункциональное тестирование — это проверка того, *как* работает система. Оно касается характеристик качества, не связанных напрямую с конкретными бизнес-функциями.

Ключевые виды (на основе ISO 25010):

- **Производительность (Performance):** Скорость работы, время отклика, нагрузка.
- **Надежность (Reliability):** Стабильность, устойчивость к сбоям, восстановление после ошибок.
- **Юзабилити (Usability):** Удобство использования, понятность интерфейса.
- **Безопасность (Security):** Защита от несанкционированного доступа, взломов.
- **Совместимость (Compatibility):** Работа в разных браузерах, на разных устройствах и ОС.
- **Сопровождаемость (Maintainability):** Насколько легко изменять и поддерживать код.

2. Дополнительная информация:

- Часто нефункциональные требования (например, "система должна выдерживать 1000 пользователей") забывают или недооценивают, хотя они критичны для успеха продукта.
- Принцип 7 ("Заблуждение об отсутствии ошибок", Лекция 2) как раз об этом: программа может быть идеальной функционально, но неудобной или медленной, и пользователи уйдут.
- Эти проверки часто выполняются специальными инструментами (JMeter для нагрузки, Lighthouse для производительности).

3. Пример

- Калькулятор скидок (Лекция 1) идеально считает, но если страница грузится 10 секунд (провал по производительности) или на мобильном

телефоне кнопки съезжают (провал по совместимости и юзабилити), то продукт плохой, несмотря на правильные расчеты.

Вопрос 17. Тестирование «белым ящиком»

Тестирование «белым ящиком» (White-box testing) — это метод тестирования, при котором тестировщик имеет доступ к внутренней структуре, коду и архитектуре программы. Его еще называют "прозрачным ящиком" или "стеклянным ящиком".

Цели (что должно быть проверено в коде):

1. Все независимые пути выполнения проверены хотя бы один раз (Path coverage).
2. Все логические ветвления (if/else, switch) проверены на TRUE и FALSE (Decision coverage).
3. Все циклы проверены на границах и в пределах рабочих значений.

Техники покрытия (от простого к сложному):

- **Statement coverage:** Проверка, что каждая строка кода была выполнена.
- **Decision coverage:** Проверка истинности и ложности каждого условия.
- **Condition coverage:** Проверка каждого отдельного подусловия.
- **Multiple-condition coverage:** Проверка всех комбинаций условий.
- **Path coverage:** Проверка всех возможных путей в коде (самое сложное).

2. Дополнительная информация:

- **Преимущества:** Можно начать тестировать рано (код еще без интерфейса), очень тщательное покрытие.
- **Недостатки:** Требуется глубоких знаний кода, сложно поддерживать автотесты при частых изменениях.
- Тестирование циклов по правилам: не выполняется, выполняется 1 раз, m раз ($m < n$), n раз.
- Формула покрытия: $Tcov = (Ltc / Lcode) * 100\%$.

3. Пример

- У нас есть функция `calculate_discount(price, user_type, promo)` (Лекция 1). Тест белым ящиком: мы смотрим в код и убеждаемся, что наши тесты проходят по всем веткам `if/else`: 1) `user_type == "guest"`, 2) `user_type == "registered" && promo == "SALE10"`, 3) `user_type == "registered" && promo == "SALE20"`, 4) ошибка, 5) ошибка. Каждая ветка проверена.
-

Вопрос 18. Тестирование «черным ящиком»

Тестирование «черным ящиком» (Black-box testing) — это метод тестирования, при котором внутренняя структура программы неизвестна. Мы видим только входные данные и выходные результаты, как "черный ящик".

Основные техники (все были в Лекции 1):

1. **Эквивалентное разбиение (Equivalence Partitioning):** Входные данные делятся на классы (группы), которые должны обрабатываться одинаково. Для теста берется одно значение из каждого класса.
2. **Анализ граничных значений (Boundary Value Analysis):** Проверяются значения на границах классов эквивалентности и чуть за ними (всего три точки: на границе, чуть меньше, чуть больше).
3. **Анализ причинно-следственных связей (Cause-Effect Graphing):** Строится таблица решений, где перебираются комбинации условий (причин) и их результатов (следствий).

2. Дополнительная информация:

- **Преимущества:** Не требует знания кода, может проводиться независимым тестировщиком, отражает реальное поведение пользователя.
- **Недостатки:** Может оставить непроверенными внутренние пути кода (например, скрытые ошибки), требует хорошо проработанной документации.

- Техники "черного ящика" помогают не тестировать всё подряд, а выбрать наиболее важные и репрезентативные тесты.

3. Пример

- В задаче со скидкой (Лекция 1): мы не знаем, как внутри написана функция, но по требованиям выделяем классы:
 - Правильные классы: price от 1 до 10000, user_type = "guest" или "registered", promo = "", "SALE10", "SALE20".
 - Неправильные классы: price < 1, price > 10000, user_type = "admin" (невалидный), promo = "SALE50" (невалидный).
- Проверяем по одному представителю из каждого класса и граничные точки (1, 0, 10000, 10001).

Вопрос 19. Usability-тестирование

Usability-тестирование (тестирование удобства использования / юзабилити) — это вид нефункционального тестирования, направленный на оценку того, насколько продукт удобен, понятен и приятен для конечного пользователя.

Оно оценивает:

- **Обучаемость:** Как легко пользователь учится работать с продуктом.
- **Эффективность:** Как быстро пользователь выполняет свои задачи.
- **Запоминаемость:** Может ли пользователь вспомнить, как пользоваться продуктом после перерыва.
- **Ошибки:** Сколько ошибок делает пользователь, насколько они серьезны, как легко их исправить.
- **Удовлетворенность:** Насколько приятен пользователю интерфейс и взаимодействие.

2. Дополнительная информация:

- Это крайне важная часть тестирования, так как даже функционально идеальный продукт может быть провальным, если его интерфейс неудобен (Принцип 7, Лекция 2).
- Обычно проводится с привлечением реальных пользователей (или фокус-групп) в начале разработки (прототипы) или на готовом продукте.
- Часто включает в себя наблюдение за пользователем ("подумай вслух"), A/B-тестирование, опросы.

3. Пример

- В тест-кейсах (Лекция 6) проверяется, что происходит после нажатия кнопки "Sign in". Usability-тест проверит, насколько заметна эта кнопка на странице, понятен ли текст на ней, видит ли пользователь, что он авторизован (попадает ли он в личный кабинет без лишних кликов, видит ли приветствие).

Вопрос 20. Виды заглушек в автоматизированном тестировании

В автоматизированном тестировании **заглушки (Test Doubles)** — это объекты-заменители, которые используются вместо реальных компонентов системы (например, базы данных, внешнего API, платежной системы) для изолированного тестирования. Это позволяет сделать тесты быстрыми, стабильными и независимыми.

Основные виды заглушек:

1. **Stub (Стаб):** Самый простой тип. Возвращает жестко заданный (фиксированный) ответ на определенный вызов. Не имеет логики.
 - *Пример:* Заглушка, которая всегда возвращает True при проверке пароля.
2. **Mock (Мок):** Более умный тип. Не только возвращает ответ, но и проверяет, был ли вызван метод, с какими параметрами и сколько раз.

- *Пример:* Проверка, что метод `sendEmail()` был вызван ровно 1 раз после регистрации.
3. **Fake (Фейк):** Упрощенная рабочая реализация. Имеет внутреннюю логику, но выполняет те же функции, что и реальный объект, только в упрощенном виде.
 - *Пример:* In-memory база данных вместо реальной PostgreSQL.
 4. **Dummy (Пустышка):** Передается, но никогда не используется. Нужен только для компиляции или заполнения параметров.
 5. **Spy (Шпион):** Фактически, это реальный объект, но с возможностью записывать и проверять историю вызовов.

2. Дополнительная информация:

- Использование заглушек — ключевая практика в **unit-тестировании** (Лекция 7). Например, при тестировании функции, вызывающей внешний сервис, мы используем **Mock** для этого сервиса, чтобы не зависеть от его доступности.
- Важно не путать: **Stub** — про возврат данных, **Mock** — про проверку поведения.

3. Пример

- Представьте, что мы тестируем функцию `apply_promo_code(user_id, promo_code)` (Лекция 1), которая для работы обращается к внешнему API для проверки промокода.
- Чтобы не зависеть от реального API и ускорить тест, мы создаем **MockPromoAPI**, который для кода "SALE10" возвращает ответ "valid = true", а для "SALE99" — "valid = false". Это позволяет быстро проверить 100 разных промокодов без реальных сетевых запросов.